AMITY INSTITUTE OF INFORMATION TECHNOLOGY

Course Code : CSE2052

"INTRODUCTION TO ARTIFICAL INTELLIGENCE"

LABORATORY RECORD

Fall Semester – 2025-26

SUBMITTED TO:	**SUBMITTED BY:**
DR MADHU BHAN	**ANANYA B**
	A86904824021
	BCA

INDEX

Sl. No.	Date	Program	Page No.
1	24/09/2025	SIMPLE AGENT	3
2	24/09/2025	SIMPLE AGENT	4
3	08/10/2025	PROBLEM REPRESENTATION	5
4	08/10/2025	SIMPLE AGENT	6
5	15/10/2025	GOAL BASED AGENT	7-8
6	29/10/2025	BREADTH FIRST SEARCH	9-10
7	12/11/2025	DEPTH FIRST SEARCH	11-13
8	19/11/2025	PROPOSITIONAL LOGIC	14-15
9	19/11/2025	PREDICATE LOGIC	16-17
10	26/11/2025	STATISTICAL MEASURES	18
11	26/11/2025	DATASET AND FEATURES	19
12	03/12/2025	MISSING VALUE HANDLING	20
13	10/12/2025	OUTLIER DETECTION	21-22
14	10/12/2025	TRAIN TEST SPLIT	23
15	24/12/2025	MIN MAX NORMALIZATION	24
16	31/12/2025	LINEAR REGRESSION	25
17	07/01/2026	BEST FIT REGRESSION LINE	26-27
18	21/01/2026	DECISION TREE	28-29

Sl. No.	Date	Program	Page No.
19	21/01/2026	LOGISTIC REGRESSION	30
20	28/01/2026	IMPLEMENT ALGORITHM FOR K-MEANS	• 31`

Program 1

Write a program where the agent decides an action based on conditions.
Example: If it is "sunny" → print "Wear sunglasses"; If it is "rainy" → print "Carry umbrella".

INPUT:

```
weather = input("Enter the weather (sunny, rainy, cloudy, cold): ").lower()

if weather == "sunny":

    print("wear sunglasses")

elif weather == "rainy":

    print("carry umbrella")

elif weather == "cloudy":

    print("It might rain, better take a jacket")

elif weather == "cold":

    print("wear a warm coat")

else:

    print("No specific advice, enjoy your day!!")
```

OUTPUT:

```
In [10]: weather = input("Enter the weather (sunny, rainy, cloudy, cold): ").lower()

         if weather == "sunny":
             print("wear sunglasses")
         elif weather == "rainy":
             print("carry umbrella")
         elif weather == "cloudy":
             print("It might rain, better take a jacket")
         elif weather == "cold":
             print("wear a warm coat")
         else:
             print("No specific advice, enjoy your day!!")

         Enter the weather (sunny, rainy, cloudy, cold): happy
         No specific advice, enjoy your day!!
```

Program 2

Write a program where the agent decides an action based on temperature. Example: If temp > 30 → print "Drink plenty of water"; else if temp > 20 → print "Weather is pleasant"; else if temp > 10 → print "Wear a jacket".

INPUT:

```
temp = int(input("Enter temperature: "))
if temp > 30:
    print("Drink plenty of water")
elif temp > 20:
    print("Weather is pleasant")
elif temp > 10:
    print("Wear a jacket")
else:
    print("It's very cold, stay warm")
```

OUTPUT:

Program 3

Write a program for 8-Puzzle problem state representation (8-puzzle new). Display as 3×3 matrix.
Compare initial state and goal state and if equal display "the two states are same", else display "the two states are different".

INPUT:

```
initial_state = [1, 2, 3, 4, 5, 6, 7, 8, "_"]

goal_state = [1, 2, 3, 4, 5, 6, 7, 8, "_"]

def display_state(state):

    for i in range(0, len(state), 3):

        print(state[i:i+3])

    print()

    print("Initial State:")

display_state(initial_state)

print("Goal State:")

display_state(goal_state)

if initial_state == goal_state:

    print("The states are the same.")

else:

    print("The states are different.")
```

OUTPUT:

```
In [7]: initial_state = [1, 2, 3, 4, 5, 6, 7, 8, "_"]
        goal_state = [1, 2, 3, 4, 5, 6, 7, 8, "_"]
        def display_state(state):
            for i in range(0, len(state), 3):
                print(state[i:i+3])
            print()
            print("Initial State:")
        display_state(initial_state)

        print("Goal State:")
        display_state(goal_state)
        if initial_state == goal_state:
            print("The states are the same.")
        else:
            print("The states are different.")

        [1, 2, 3]
        [4, 5, 6]
        [7, 8, '_']

        Initial State:
        Goal State:
        [1, 2, 3]
        [4, 5, 6]
        [7, 8, '_']

        Initial State:
        The states are the same.
```

Program 4

Write a program to simulate a simple Vacuum Cleaner Agent that cleans two locations (A and B).
The agent should check if the current location is dirty — if so clean it; otherwise move to the next location.
Finally, display the initial and final environment.

INPUT:

```
room_status = input("Enter the status of the room (c/d): ").lower()

if room_status == "c":

    print("Clean it properly")

elif room_status=="d":

   print("move to next roon")

else:

   print("Invalid input. Please enter 'dirty' or 'clean'.")
```

OUTPUT:

```
In [3]: room_status = input("Enter the status of the room (c/d): ").lower()
        if room_status == "c":
            print("Clean it properly")
        elif room_status=="d":
           print("move to next roon")
        else:
           print("Invalid input. Please enter 'dirty' or 'clean'.")

        Enter the status of the room (c/d): d
        move to next roon
```

Program 5

Write a program to simulate an Autonomous Car Agent that can make driving decisions based on environmental conditions such as traffic light color, obstacle detection, and road condition.
The car should decide whether to move, stop, or slow down depending on the situation.

INPUT:

```
def autonomous_car_agent(traffic_light, obstacle, road_condition):
    print("Traffic Light:", traffic_light)
    print("Obstacle Detected:", obstacle)
    print("Road Condition:", road_condition)
    if traffic_light.lower() == "red":
        action = "Stop the car"
    elif traffic_light.lower() == "yellow":
        action = "Slow down and prepare to stop"
    elif obstacle.lower() == "yes":
        action = "Stop — obstacle ahead"
    elif road_condition.lower() == "slippery":
        action = "Drive slowly and carefully"
    elif traffic_light.lower() == "green" and obstacle.lower() == "no":
        action = "Move forward safely"
    else:
        action = "Stay alert and maintain current state"
    print("Decision:", action)
print("=== Scenario 1 ===")
autonomous_car_agent("Red", "No", "Clear")


print("\n=== Scenario 2 ===")
autonomous_car_agent("Green", "Yes", "Clear")
autonomous_car print("\n=== Scenario 3 ===")
_agent("Green", "No", "Slippery")
```

OUTPUT:

```
In [2]: def autonomous_car_agent(traffic_light, obstacle, road_condition):
            print("Traffic Light:", traffic_light)
            print("Obstacle Detected:", obstacle)
            print("Road Condition:", road_condition)
            if traffic_light.lower() == "red":
                action = "Stop the car"
            elif traffic_light.lower() == "yellow":
                action = "Slow down and prepare to stop"
            elif obstacle.lower() == "yes":
                action = "Stop – obstacle ahead"
            elif road_condition.lower() == "slippery":
                action = "Drive slowly and carefully"
            elif traffic_light.lower() == "green" and obstacle.lower() == "no":
                action = "Move forward safely"
            else:
                action = "Stay alert and maintain current state"

            print("Decision:", action)
        print("=== Scenario 1 ===")
        autonomous_car_agent("Red", "No", "Clear")

        print("\n=== Scenario 2 ===")
        autonomous_car_agent("Green", "Yes", "Clear")

        print("\n=== Scenario 3 ===")
        autonomous_car_agent("Green", "No", "Slippery")
```


```
In [2]: def autonomous_car_agent(traffic_light, obstacle, road_condition):
            print("Traffic Light:", traffic_light)
            print("Obstacle Detected:", obstacle)
            print("Road Condition:", road_condition)
            if traffic_light.lower() == "red":
                action = "Stop the car"
            elif traffic_light.lower() == "yellow":
                action = "Slow down and prepare to stop"
            elif obstacle.lower() == "yes":
                action = "Stop – obstacle ahead"
            elif road_condition.lower() == "slippery":
                action = "Drive slowly and carefully"
            elif traffic_light.lower() == "green" and obstacle.lower() == "no":
                action = "Move forward safely"
            else:
                action = "Stay alert and maintain current state"

            print("Decision:", action)
        print("=== Scenario 1 ===")
        autonomous_car_agent("Red", "No", "Clear")

        print("\n=== Scenario 2 ===")
        autonomous_car_agent("Green", "Yes", "Clear")

        print("\n=== Scenario 3 ===")
        autonomous_car_agent("Green", "No", "Slippery")
```


```
=== Scenario 1 ===
Traffic Light: Red
Obstacle Detected: No
Road Condition: Clear
Decision: Stop the car

=== Scenario 2 ===
Traffic Light: Green
Obstacle Detected: Yes
Road Condition: Clear
Decision: Stop – obstacle ahead

=== Scenario 3 ===
Traffic Light: Green
Obstacle Detected: No
Road Condition: Slippery
Decision: Drive slowly and carefully
```

Program 6

Write a program that accepts an adjacency list of a graph as input and performs Breadth First Search (BFS) starting from a given source node. The program should display the order in which the nodes are visited.

INPUT:

```
n = int(input("Enter the number of nodes: "))
nodes = []
print("Enter the node names:")
for i in range(n):
    name = input(f"Node {i}: ")
    nodes.append(name)
graph = {}
print("\nEnter the adjacent nodes for each node (space-separated). If none, press Enter:")
for node in nodes:
    neighbors = input(f"Neighbors of {node}: ").split()
    graph[node] = neighbors
print("\nAdjacency List Representation:")
for node in graph:
    print(node, "->", ", ".join(graph[node]))
start = input("\nEnter the starting node for BFS: ")
visited = []
queue = []
visited.append(start)
queue.append(start)

print("\nBFS Traversal Order:", end=" ")
while queue:
    current = queue.pop(0)
    print(current, end=" ")
```

```
    for neighbor in graph[current]:
        if neighbor not in visited:
            visited.append(neighbor)
            queue.append(neighbor)

print()
```

OUTPUT:

```
In [2]: n = int(input("Enter the number of nodes: "))

        nodes = []
        print("Enter the node names:")
        for i in range(n):
            name = input(f"Node {i}: ")
            nodes.append(name)

        graph = {}
        print("\nEnter the adjacent nodes for each node (space-separated). If none, press Enter:")
        for node in nodes:
            neighbors = input(f"Neighbors of {node}: ").split()
            graph[node] = neighbors

        print("\nAdjacency List Representation:")
        for node in graph:
            print(node, "->", ", ".join(graph[node]))

        start = input("\nEnter the starting node for BFS: ")

        visited = []
        queue = []

        visited.append(start)
        queue.append(start)

        print("\nBFS Traversal Order:", end=" ")

        while queue:
            current = queue.pop(0)
            print(current, end=" ")

            for neighbor in graph[current]:
                if neighbor not in visited:
                    visited.append(neighbor)
                    queue.append(neighbor)

        print()
```

```
Enter the number of nodes: 6
Enter the node names:
Node 0: a
Node 1: b
Node 2: c
Node 3: d
Node 4: e
Node 5: f

Enter the adjacent nodes for each node (space-separated). If none, press Enter:
Neighbors of a: b c
Neighbors of b: a c d e
Neighbors of c: a b f
Neighbors of d: b e
Neighbors of e: b d
Neighbors of f: c

Adjacency List Representation:
a -> b, c
b -> a, c, d, e
c -> a, b, f
d -> b, e
e -> b, d
f -> c

Enter the starting node for BFS: a

BFS Traversal Order: a b c d e f
```

Program 7

Write a program that accepts an adjacency list of a graph as input and performs Depth First Search (DFS) starting from a given source node. The program should display the order in which the nodes are visited.

INPUT:

```
n = int(input("Enter the number of nodes: "))
nodes = []

print("Enter the node names:")
for i in range(n):
    name = input(f"Node {i+1} name: ")
    nodes.append(name)

graph = {}

print("\nEnter the adjacent nodes for each node (separated by space):")
for node in nodes:
    neighbors = input(f"Neighbors of {node}: ").replace(",", " ").split()
    graph[node] = neighbors

print("\nAdjacency List Representation:")
for node in graph:
    print(node, "->", ", ".join(graph[node]))

start = input("\nEnter the starting node for DFS: ")

visited = []
stack = [start]

print("\nDFS Traversal Order:", end=" ")
```

```
while stack:
    node = stack.pop()      # pop from end → DFS
    if node not in visited:
        print(node, end=" ")
        visited.append(node)

        # push neighbors in reverse so left-most is processed first
        for neighbor in reversed(graph[node]):
            if neighbor not in visited:
                stack.append(neighbor)

print()
```

OUTPUT:

```
In [1]: n = int(input("Enter the number of nodes: "))
        nodes = []

        print("Enter the node names:")
        for i in range(n):
            name = input(f"Node {i+1} name: ")
            nodes.append(name)

        graph = {}

        print("\nEnter the adjacent nodes for each node (separated by space):")
        for node in nodes:
            neighbors = input(f"Neighbors of {node}: ").replace(",", " ").split()
            graph[node] = neighbors

        print("\nAdjacency List Representation:")
        for node in graph:
            print(node, "->", ", ".join(graph[node]))

        start = input("\nEnter the starting node for DFS: ")

        visited = []
        stack = [start]

        print("\nDFS Traversal Order:", end=" ")
        while stack:
            node = stack.pop()
            if node not in visited:
                print(node, end=" ")
                visited.append(node)

                for neighbor in reversed(graph[node]):
                    if neighbor not in visited:
                        stack.append(neighbor)

        print()
```

```
Enter the number of nodes: 6
Enter the node names:
Node 1 name: a
Node 2 name: b
Node 3 name: c
Node 4 name: d
Node 5 name: e
Node 6 name: f

Enter the adjacent nodes for each node (separated by space):
Neighbors of a: b,c
Neighbors of b: a,d,e
Neighbors of c: a,f
Neighbors of d: b
Neighbors of e: b
Neighbors of f: c

Adjacency List Representation:
a -> b, c
b -> a, d, e
c -> a, f
d -> b
e -> b
f -> c

Enter the starting node for DFS: a

DFS Traversal Order: a b d e c f
```

Program 8

Write a program to evaluate basic propositional logic expressions.
The user should enter truth values (True/False) for propositions P and Q.
The program should compute and display the results of the following logical operations:
P AND Q
P OR Q
NOT P
P → Q (Implication)

INPUT:

```
print("Enter the truth values for P and Q (True/False):")

P_input = input("Enter P (True/False): ")

Q_input = input("Enter Q (True/False): ")


# Convert string input to boolean

if P_input == "True":

    P = True

else:

    P = False


if Q_input == "True":

    Q = True

else:

    Q = False


# Logical operations

and_result = P and Q

or_result = P or Q

not_result = not P

implication_result = (not P) or Q

biconditional_result = (P and Q) or ((not P) and (not Q))
```

```
# Display results
print("\n----Results----")
print("P and Q     =", and_result)
print("P or Q      =", or_result)
print("not P       =", not_result)
print("P -> Q      =", implication_result)
print("P <-> Q     =", biconditional_result)
```

OUTPUT:

```
In [1]: print("Enter the truth values for P and Q (True/False):")
        P_input = input("Enter P (True/False): ")
        Q_input = input("Enter Q (True/False): ")
        if P_input == "True":
            P = True
        else:
            P = False

        if Q_input == "True":
            Q = True
        else:
            Q = False
        and_result = P and Q
        or_result = P or Q
        not_result = not P
        implication_result = (not P) or Q
        biconditional_result = (P and Q) or ((not P) and (not Q))
        print("\n----Results----")
        print("P and Q     =", and_result)
        print("P or Q      =", or_result)
        print("not P       =", not_result)
        print("P -> Q      =", implication_result)
        print("P <-> Q     =", biconditional_result)

        Enter the truth values for P and Q (True/False):
        Enter P (True/False): True
        Enter Q (True/False): False

        ----Results----
        P and Q     = False
        P or Q      = True
        not P       = False
        P -> Q      = False
        P <-> Q     = False
```

Program 9

Write a program that stores a few predicate facts (like parent, brother, teacher) using simple lists.
The user will enter a query such as: parent(John, Mary).
The program should check whether this fact exists in the list and print: Fact exists / Fact does NOT exist.

INPUT:

```
parent_facts=[("John"."Mary"),("Ravi","Amit")]

brother_facts=[("Amit"."Rohan"),("Rohan","Amit")]

teacher_facts=[("Seema"."Class5"),("Arun","Class3")]

print("Enter a query like: parent(John,Mary)")

quer= input("Query: ")

parts= query.split("(")

predicate=parts[0]

args = pary[1].replace(")","")

arg1, arg2 = args.split(",")

arg1= arg1.strip()

arg2= arg2.strip()

fact_exists = False

if predicate == "parent":

   facts = parent_facts

elif predicate == "brother":

   facts = brother_facts

elif predicate == "teacher":

   facts = teacher_facts

else:

   print("Unknown predicate:", predicate)

   exit()

if (arg1, arg2) in facts:
```

```
    fact_exists = True
if fact_exists:
    print(f"YES, {predicate}({arg1},{arg2}) is TRUE")
else:
    print(f"NO, {predicate}({arg1},{arg2}) is FALSE")
```

OUTPUT:

```
In [3]: parent_facts = [("John", "Mary"), ("Ravi", "Amit")]
        brother_facts = [("Amit", "Rohan"), ("Rohan", "Amit")]
        teacher_facts = [("Seema", "Class5"), ("Arun", "Class3")]
        print("Enter a query like: parent(John,Mary)")
        query = input("Query: ")
        parts = query.split("(")
        predicate = parts[0].strip()
        args = parts[1].replace(")", "")
        arg1, arg2 = args.split(",")
        arg1 = arg1.strip()
        arg2 = arg2.strip()
        fact_exists = False
        if predicate == "parent":
            if (arg1, arg2) in parent_facts:
                fact_exists = True
        elif predicate == "brother":
            if (arg1, arg2) in brother_facts:
                fact_exists = True
        elif predicate == "teacher":
            if (arg1, arg2) in teacher_facts:
                fact_exists = True
        else:
            print("Unknown predicate:", predicate)
            exit()
        if fact_exists:
            print(f"YES, {predicate}({arg1},{arg2}) is TRUE")
        else:
            print(f"NO, {predicate}({arg1},{arg2}) is FALSE")

        Enter a query like: parent(John,Mary)
        Query: parent(Rohan,Amit)
        NO, parent(Rohan,Amit) is FALSE
```

Program 10

Find Mean, Median, Mode, and Standard Deviation of a given dataset:
speed = [99, 86, 87, 88, 111, 86, 103, 87, 94, 78, 77, 85, 86]

INPUT:

```
import numpy

from scipy import stats

speed = [99, 86, 87, 88, 111, 86, 103, 87, 94, 78, 77, 85, 86]

x = numpy.mean(speed)

y = numpy.median(speed)

z = stats.mode(speed)

p = numpy.std(speed)

print("mean is", x)

print("median is", y)

print("mode is", z)

print("std is", p)
```

OUTPUT:

```
In [2]: import numpy
        from scipy import stats
        speed = [99,86,87,88,111,86,103,87,94,78,77,85,86]
        x = numpy.mean(speed)
        y = numpy.median(speed)
        z= stats.mode(speed)
        p = numpy.std(speed)
        print( "mean is",x)
        print("median is",y)
        print("mode is",z)
        print("std is",p)

        mean is 89.76923076923077
        median is 87.0
        mode is ModeResult(mode=86, count=3)
        std is 9.258292301032677
```

Program 11

Write a program to demonstrate the concept of a dataset and features. Create a small dataset, identify the feature and target values, and display them separately.

INPUT:

```
import numpy as np

# Step 1: Create a simple dataset

x = np.array([1, 2, 3, 4])      # Feature (1-D)

y = np.array([10, 20, 30, 40])  # Target / Label

print("Feature before reshape (1-D):")

print(x)

print("Shape:", x.shape)

# Step 2: Reshape feature into 2-D

x = x.reshape(-1, 1)

print("\nFeature after reshape (2-D):")

print(x)

print("Shape:", x.shape)
```

OUTPUT:

```
In [2]: import numpy as np
        # Step 1: Create a simple dataset
        x = np.array([1, 2, 3, 4])      # Feature (1-D)
        y = np.array([10, 20, 30, 40])  # Target
        print("Feature before reshape (1-D):")
        print(x)
        print("Shape:", x.shape)
        # Step 2: Reshape feature into 2-D
        x = x.reshape(-1, 1)
        print("\nFeature after reshape (2-D):")
        print(x)
        print("Shape:", x.shape)

        Feature before reshape (1-D):
        [1 2 3 4]
        Shape: (4,)

        Feature after reshape (2-D):
        [[1]
         [2]
         [3]
         [4]]
        Shape: (4, 1)
```

Program 12

Design a program to demonstrate the use of SimpleImputer class for filling the missing values.

INPUT:

```
In [9]: from sklearn.impute import SimpleImputer
        import numpy as np
        from scipy import stats
        data = np.array([
            [10, 2],
            [15, np.nan],
            [np.nan, 4],
            [20, 5]
        ])
        print("Original Data:")
        print(data)
        imputer = SimpleImputer(strategy='most_frequent')
        imputed_data = imputer.fit_transform(data)
        print("\nData After Imputation:")
        print(imputed_data)
```

OUTPUT:

```
In [9]: from sklearn.impute import SimpleImputer
        import numpy as np
        from scipy import stats
        data = np.array([
            [10, 2],
            [15, np.nan],
            [np.nan, 4],
            [20, 5]
        ])
        print("Original Data: ")
        print(data)
        Imputer = SimpleImputer(strategy='most_frequent')
        imputed_data = Imputer.fit_transform(data)
        print("\nData After Imputation:" )
        print(imputed_data)

        Original Data:
        [[10.  2.]
         [15. nan]
         [nan  4.]
         [20.  5.]]

        Data After Imputation:
        [[10.  2.]
         [15.  2.]
         [10.  4.]
         [20.  5.]]
```

Program 13

Write a program to find outliers using Z-score.

INPUT:

```
import numpy as np

from scipy import stats

# Step 1: Create a dataset

data = [10, 12, 14, 15, 16, 18, 20, 22, 24, 100]

# Step 2: Calculate Z-scores

z_scores = stats.zscore(data)

# Step 3: Set threshold

threshold = 3

# Step 4: Find outliers

outliers = []

for i in range(len(data)):

    if abs(z_scores[i]) > threshold:

        outliers.append(data[i])

# Step 5: Display results

print("Data:", data)

print("Z-scores:", z_scores)

print("Outliers:", outliers)
```

OUTPUT:

```
In [6]: import numpy as np
        from scipy import stats

        # Step 1: Create a dataset
        data = [10, 12, 14, 15, 16, 18, 20, 22, 24, 100]

        # Step 2: Calculate Z-scores
        z_scores = stats.zscore(data)

        # Step 3: Set threshold
        threshold = 3

        # Step 4: Find outliers
        outliers = []
        for i in range(len(data)):
            if abs(z_scores[i]) > threshold:
                outliers.append(data[i])

        # Step 5: Display results
        print("Data:", data)
        print("Z-scores:", z_scores)
        print("Outliers:", outliers)

        Data: [10, 12, 14, 15, 16, 18, 20, 22, 24, 100]
        Z-scores: [-0.59665155 -0.51762485 -0.43859816 -0.39908481 -0.35957146 -0.28054477
         -0.20151807 -0.12249138 -0.04346468  2.95954972]
        Outliers: []
```

Program 14

Design a program to demonstrate how a dataset is divided into training data and testing data using the train_test_split() function from the scikit-learn library.

INPUT:

```
import numpy as np
from sklearn.model_selection import train_test_split
x = np.array([800, 1000, 1200, 1500, 1800]).reshape(-1, 1)
y = np.array([120, 150, 180, 210, 250])
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42
)
print("x_train:", x_train)
print("y_train:", y_train)
print("x_test:", x_test)
print("y_test:", y_test)
```

OUTPUT:

```
import numpy as np
from sklearn.model_selection import train_test_split

x = np.array([800,1000,1200,1500,1800]).reshape(-1,1)
y = np.array([120,150,180,210,250])
x_train,x_test,y_train,y_test = train_test_split(
    x, y, test_size=0.2, random_state=42)

print("x_train:",x_train)
print("y_train:",y_train)
print("x_test:", x_test)
print("y_test:", y_test)
```

```
x_train: [[1800]
 [1200]
 [ 800]
 [1500]]
y_train: [250 180 120 210]
x_test: [[1000]]
y_test: [150]
```

Program 15

Given the following dataset representing marks scored by students:
[10, 12, 14, 15, 16, 18, 20, 22, 24]
Write a program to perform Min-Max Normalization to scale the data between 0 and 1.

INPUT:

```
import numpy as np

data = np.array([10, 12, 14, 15, 16, 18, 20, 22, 24])

data_min = np.min(data)

data_max = np.max(data)

normalized_data = (data - data_min) / (data_max - data_min)

print("Original Data:", data)

print("Minimum Value:", data_min)

print("Maximum Value:", data_max)

print("Normalized Data:", normalized_data)
```

OUTPUT:

```
In [5]: import numpy as np
        # Given dataset
        data = np.array([10, 12, 14, 15, 16, 18, 20, 22, 24])
        # Find min and max
        data_min = np.min(data)
        data_max = np.max(data)
        # Apply Min-Max Normalization
        normalized_data = (data - data_min) / (data_max - data_min)
        # Display results
        print("Original Data:", data)
        print("Minimum Value:", data_min)
        print("Maximum Value:", data_max)
        print("Normalized Data:", normalized_data)

        Original Data: [10 12 14 15 16 18 20 22 24]
        Minimum Value: 10
        Maximum Value: 24
        Normalized Data: [0.         0.14285714 0.28571429 0.35714286 0.42857143 0.57142857
         0.71428571 0.85714286 1.        ]

In [ ]:
```

Program 16

Design a simple program to implement Simple Linear Regression using Python and scikit-learn to predict house prices based on house size.

INPUT:

```
import numpy as np

X = np.array([500, 800, 1000, 1200, 1500])

y = np.array([20, 30, 40, 50, 65])

mean_x = np.mean(X)

mean_y = np.mean(y)

m = sum((X - mean_x) * (y - mean_y)) / sum((X - mean_x) ** 2)

c = mean_y - m * mean_x

size = int(input("Enter house size in sqft: "))

price = m * size + c

print("Predicted House Price =", round(price, 2), "lakhs")
```

OUTPUT:

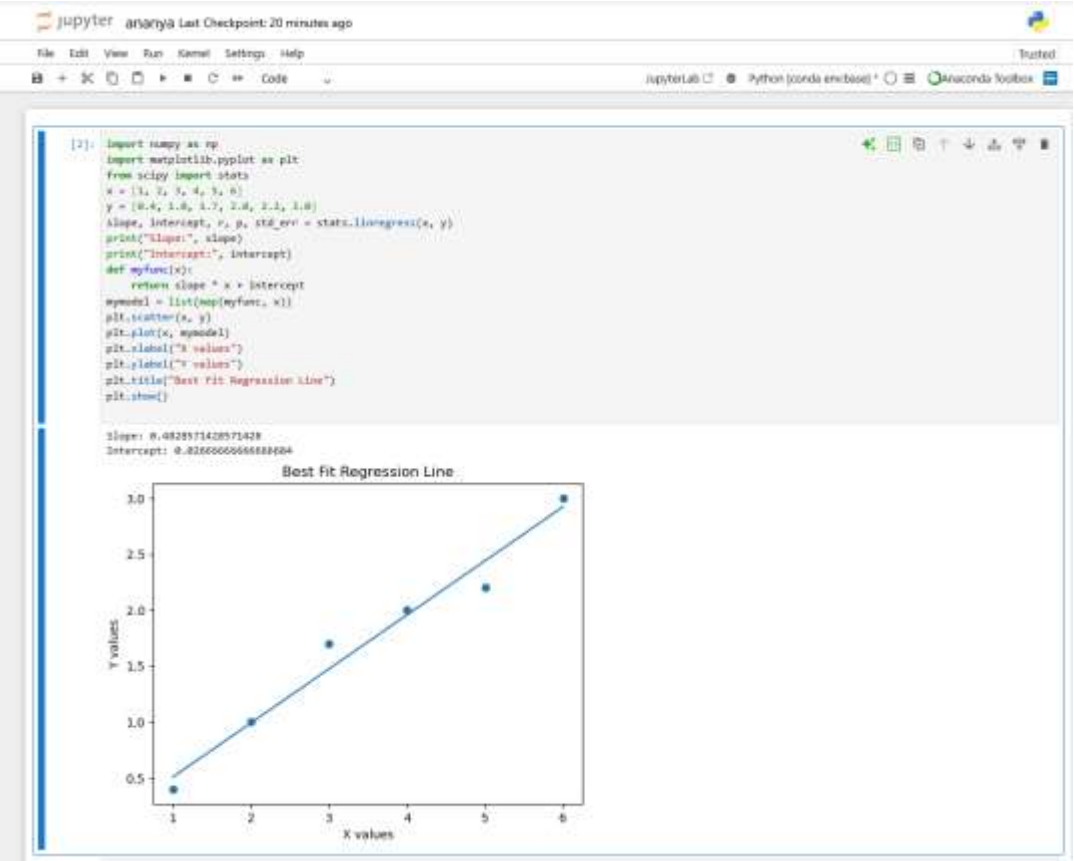
Program 17

Design and Develop a Python program to draw the best fit regression line.

INPUT:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
x = [1, 2, 3, 4, 5, 6]
y = [0.4, 1.0, 1.7, 2.0, 2.2, 3.0]
slope, intercept, r, p, std_err = stats.linregress(x, y)
print("Slope:", slope)
print("Intercept:", intercept)
def myfunc(x):
    return slope * x + intercept
mymodel = list(map(myfunc, x))
plt.scatter(x, y)
plt.plot(x, mymodel)
plt.xlabel("X values")
plt.ylabel("Y values")
plt.title("Best Fit Regression Line")
plt.show()
```

OUTPUT:


jupyter ananya Last Checkpoint: 20 minutes ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python (conda env:base)
Anaconda Toolbox
[2]:
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
x = [1, 2, 3, 4, 5, 6]
y = [0.4, 1.0, 1.7, 2.0, 2.1, 3.0]
slope, intercept, r, p, std_err = stats.linregress(x, y)
print("Slope:", slope)
print("Intercept:", intercept)
def myfunc(x):
return slope * x + intercept
mymodel = list(map(myfunc, x))
plt.scatter(x, y)
plt.plot(x, mymodel)
plt.xlabel("X values")
plt.ylabel("Y values")
plt.title("Best Fit Regression Line")
plt.show()
Slope: 0.482857142857142
Intercept: 0.826666666666666
Best Fit Regression Line
3.0
2.5
2.0
1.5
1.0
0.5
Y values
1
2
3
4
5
6
X values

Program 18

Design and Develop a program that uses a Decision Tree classifier to predict diabetes outcomes.

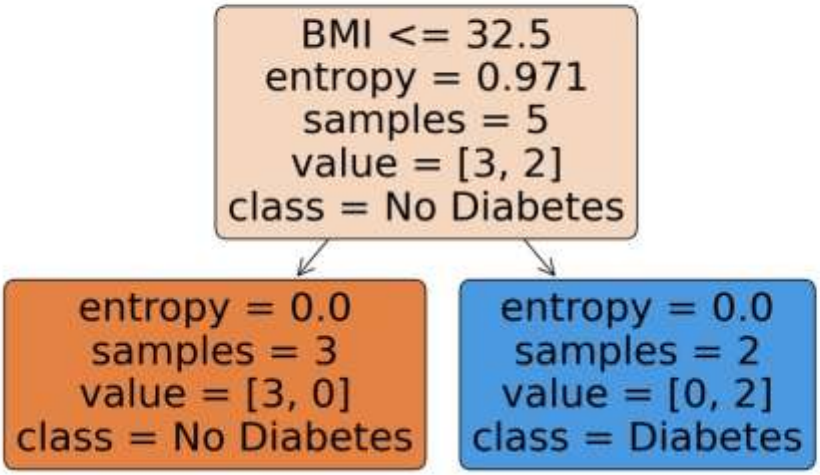
INPUT:

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
# Step 1: Dataset
# Features: [Glucose, BMI]
X = [
    [120, 30],
    [85, 25],
    [90, 28],
    [160, 35],
    [200, 40]
]
# Labels: 0 = No Diabetes, 1 = Diabetes
y = [0, 0, 0, 1, 1]
# Step 2: Create Decision Tree model using ENTROPY
model = DecisionTreeClassifier(criterion="entropy")
model.fit(X, y)
# Step 3: Visualize the tree
plt.figure(figsize=(10, 6))
plot_tree(
    model,
    feature_names=["Glucose", "BMI"],
    class_names=["No Diabetes", "Diabetes"],
    filled=True,
    rounded=True
)
plt.show()
```

OUTPUT:

```
In [1]: from sklearn.tree import DecisionTreeClassifier, plot_tree
        import matplotlib.pyplot as plt

        # Step 1: Dataset
        # Features: [Glucose, BMI]
        X = [
            [120, 30],
            [85, 25],
            [90, 28],
            [160, 35],
            [200, 40]
        ]

        # Labels: 0 = No Diabetes, 1 = Diabetes
        y = [0, 0, 0, 1, 1]

        # Step 2: Create Decision Tree model using ENTROPY
        model = DecisionTreeClassifier(criterion="entropy")
        model.fit(X, y)

        # Step 3: Visualize the tree
        plt.figure(figsize=(10, 6))
        plot_tree(
            model,
            feature_names=["Glucose", "BMI"],
            class_names=["No Diabetes", "Diabetes"],
            filled=True,
            rounded=True
        )
        plt.show()
```

```
    model,
    feature_names=["Glucose", "BMI"],
    class_names=["No Diabetes", "Diabetes"],
    filled=True,
    rounded=True
)
plt.show()
```



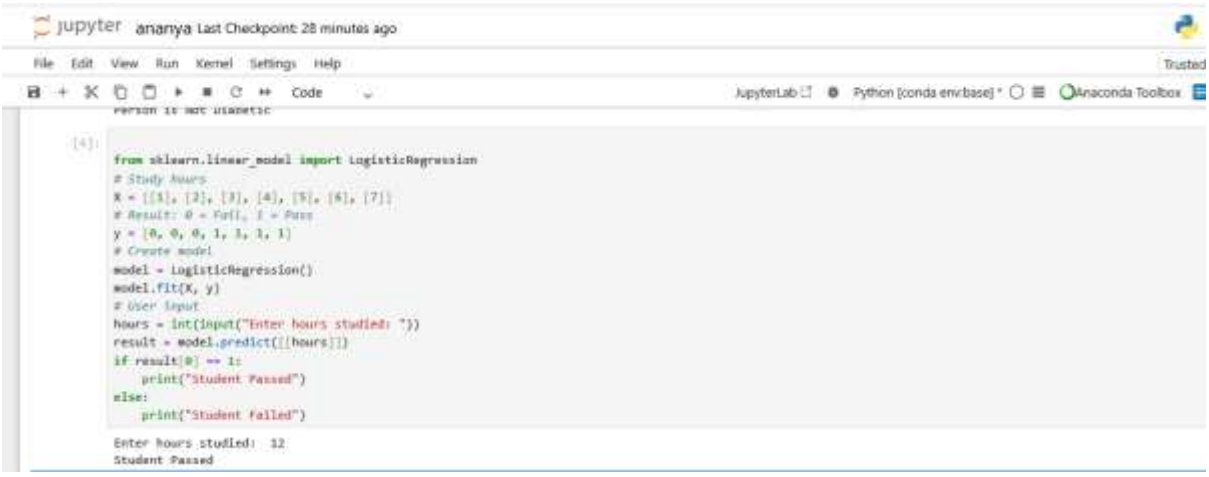
Program 19

Write a program to implement Logistic Regression for a binary classification problem that predicts whether a student passes or fails based on the number of hours studied.

INPUT:

```
from sklearn.linear_model import LogisticRegression

# Study hours

X = [[1], [2], [3], [4], [5], [6], [7]]

# Result: 0 = Fail, 1 = Pass

y = [0, 0, 0, 1, 1, 1, 1]

# Create model

model = LogisticRegression()

model.fit(X, y)

# User input

hours = int(input("Enter hours studied: "))

result = model.predict([[hours]])

if result[0] == 1:

    print("Student Passed")

else:

    print("Student Failed")
```

OUTPUT:



Program 20

Write a program to Implement algorithm for K-means.

INPUT:

```
import pandas as pd

from sklearn.cluster import KMeans

# Step 1: Load data from CSV file

data = pd.read_csv("C:/Users/student.AUBLRU/Desktop/data")

# Step 2: Create KMeans model

model = KMeans(n_clusters=2, random_state=0)

# Step 3: Fit the model

model.fit(data)

# Step 4: Display cluster labels

print("Cluster labels:")

print(model.labels_)

# Optional: View cluster centers

print("\nCluster centers:")

print(model.cluster_centers_)
```

OUTPUT:

```
In [5]: import pandas as pd
        from sklearn.cluster import KMeans

        # Step 1: Load data from CSV file
        data = pd.read_csv(r"C:\Users\student.AUBLRU\Desktop\data.csv")

        # Step 2: Keep only numeric columns (required for KMeans)
        data = data.select_dtypes(include=['number'])

        # Step 3: Create KMeans model
        model = KMeans(n_clusters=2, random_state=0, n_init=10)

        # Step 4: Fit the model
        model.fit(data)

        # Step 5: Display cluster labels
        print("Cluster labels:")
        print(model.labels_)

        # Step 6: Display cluster centers
        print("\nCluster centers:")
        print(model.cluster_centers_)

        C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1436: UserWarning: KMeans is known to have a memory leak
        on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OM
        P_NUM_THREADS=1.
          warnings.warn(

        Cluster labels:
        [1 1 1 0 0 0]

        Cluster centers:
        [[10.  2.]
         [ 1.  2.]]
```

THANK YOU